System Testing Report

Project: Web Based ML Model Evaluator

Authors:
PES2UG24CS368 – PRATHAM KUMAR - prathamkumar041104@gmail.com
PES2UG24CS324 – NIRUPAMA JAYARAMAN - jnirupama06@gmail.com
PES2UG25CS824 – SHREYAS BELLAD - shreyasbellad2006@gmail.com
PES2UG24CS339 – PARINITHA ACHARYA - parinitha.acharya2006@gmail.com

Approvals

Role	Name	Signature / Email	Date
Course Coordinator	Sheela Devi M.	sheelam@pes.edu	—

1. INTRODUCTION

1.1 Project Overview

The Web-Based ML Model Evaluator (ModelLab) is a web-based application designed to simplify and streamline the process of developing, training, and evaluating machine learning models. The platform provides an integrated environment where users can upload datasets, explore data, perform preprocessing operations, configure machine learning experiments, train models, and evaluate their performance through appropriate evaluation metrics.

The primary objective of ModelLab is to provide users with a centralized platform for experimenting with different machine learning algorithms without requiring them to independently implement the entire model development and evaluation workflow. It supports both classification and regression tasks, allowing users to select suitable algorithms based on the nature of their datasets and prediction objectives.

The application integrates frontend and backend components to provide a structured workflow for machine learning experimentation. The frontend facilitates user interaction, while the backend handles data processing, model execution, database operations, and evaluation activities.

Since ModelLab consists of multiple interconnected components and functionalities, it is important to verify not only the correctness of individual functions but also the interaction between components and the behaviour of the complete application. Functional, integration, system, and end-to-end testing are therefore performed to evaluate the application at different levels.

1.2 Introduction to Functional, Integration, System and End-to-End Testing

Software testing is an important activity in software development that verifies whether an application behaves according to its specified requirements. While unit testing focuses on individual functions and components, other testing techniques are required to validate application features, interactions between components, and complete user workflows.

The following testing approaches are used in ModelLab:

1. Functional / Component Testing

Functional testing verifies whether individual application features perform their intended operations according to the specified requirements. It focuses on validating the expected behaviour of application components by providing inputs and checking the resulting outputs.

In ModelLab, this testing approach is used to verify individual features and functionalities available within the application.

2. Integration Testing
Integration testing verifies whether different modules or components of an application communicate and work together correctly. It focuses on identifying errors that may occur when independently developed components interact with one another.
In ModelLab, integration testing focuses on validating interactions between the frontend, backend, database, and machine learning processing components.

3. System Testing
System testing evaluates the complete integrated application as a single system. It verifies whether the application satisfies its overall functional requirements and behaves correctly when its different components operate together.
In ModelLab, system testing is performed to assess the overall behaviour of the complete web-based machine learning evaluation platform.

4. End-to-End Testing
End-to-end testing verifies the complete application workflow from the user's initial interaction to the final expected outcome. It ensures that the different components and services work together throughout an entire user journey.
In ModelLab, end-to-end testing focuses on validating complete user workflows involving dataset handling, machine learning experimentation, model execution, and result evaluation.

1.3 Testing Strategy and Implementation

For evaluating the ModelLab application, a structured testing strategy was adopted to validate individual application functionalities, interactions between integrated components, overall system behaviour, and complete user workflows.

The testing implementation was organized within a dedicated tests directory. Separate test files were maintained for different testing categories to ensure modularity, clarity, and ease of execution.

The following strategy was adopted for implementing the testing activities:

1. Identification of Application Functionalities: The major features and components of ModelLab were identified based on their responsibilities and expected behaviour.
2. Test Case Design: Separate test cases were designed for functional, integration, system, and end-to-end testing to validate different aspects of the application.
3. Input Preparation: Appropriate test inputs and testing conditions were prepared to evaluate the application under different scenarios.

4. Component and Interaction Validation: Individual application features and interactions between interconnected components were examined to verify their expected behaviour.
5. Complete Workflow Validation: End-to-end test scenarios were designed to examine the complete application workflow from initial user interaction to the final output.
6. Automated Test Execution: The test cases were organized into separate testing modules and executed through the consolidated testing execution script, run_all_tests.py.
7. Result Analysis: The execution outputs were examined to identify successfully passed test cases, failures, errors, and potential issues requiring attention.

Tools and Technologies Used

Tool / Technology	Purpose
Python	Programming language used for implementing automated tests
unittest	Python testing framework used for organizing and executing test cases
Assertions	Verify actual outputs against expected results
Mocking	Simulate selected dependencies and application interactions
PowerShell	Execute testing scripts in the local environment
HTML Reports	Present testing and execution results in a readable format

The testing implementation is organized into four categories: Functional / Component Testing, Integration Testing, System Testing, and End-to-End Testing. Each category focuses on a different level of application validation, collectively helping assess the reliability and correctness of the ModelLab application.

2. FUNCTIONAL / COMPONENT TESTING

2.1 Overview of Functional / Component Test Cases

Functional / Component Testing is performed to verify whether the individual features and components of the ModelLab application operate according to their intended functionality. It focuses on validating the behaviour of application features by supplying appropriate inputs and examining their actual outputs against the expected results.

In ModelLab, functional testing is used to examine the behaviour of individual application functionalities involved in dataset handling, data processing, machine learning operations, and other application features. The test cases are organized in a dedicated Python test module named test_functional.py within the tests directory. This separation helps maintain a structured testing implementation and allows functional test cases to be executed independently.

The functional testing implementation is intended to verify the correctness of application features under different input conditions, including valid operations and relevant invalid or exceptional scenarios.

2.2 Detailed Functional Test Cases

The following 15 functional test cases are implemented to verify the behaviour of individual user-facing API features of ModelLab.

Test Case F1 – Health Endpoint Reports Service Status

- Functionality tested: /api/health
- Input: HTTP GET request to the health endpoint.
- Testing procedure: Send a request and examine the response status, response body, and content type.
- Expected output: HTTP status 200, service status ok, service name ModelLab, and JSON content type.
- Validation: Verify the response status, JSON body, and Content-Type header.

Test Case F2 – Registration Rejects Missing Name

- Functionality tested: User registration.
- Input: Registration request with an empty name.
- Testing procedure: Submit registration details with the name field left empty.
- Expected output: HTTP status 400 with an error response.
- Validation: Verify the status code and presence of the error field.

Test Case F3 – Registration Rejects Invalid Email

- Functionality tested: Email validation during registration.
- Input: Registration details containing the email invalid.
- Testing procedure: Submit registration details with an invalid email format.
- Expected output: HTTP status 400.
- Validation: Verify that the registration request is rejected.

Test Case F4 – Registration Rejects Short Password

- Functionality tested: Password validation during registration.
- Input: Registration details containing the password short.
- Testing procedure: Submit registration details with a password that does not satisfy the minimum length requirement.
- Expected output: HTTP status 400.
- Validation: Verify that the registration request is rejected.

Test Case F5 – Duplicate Registration Returns Conflict

- Functionality tested: Duplicate user registration.
- Input: Registration request using the already registered email functional@test.local.
- Testing procedure: Attempt to register another user with the same email address.
- Expected output: HTTP status 409.
- Validation: Verify that the application returns a conflict response.

Test Case F6 – Login Rejects Wrong Password

- Functionality tested: User login and password verification.
- Input: Registered email with an incorrect password.
- Testing procedure: Submit login credentials containing the wrong password.
- Expected output: HTTP status 401 with an error response.
- Validation: Verify the response status and error field.

Test Case F7 – Profile Endpoint Requires Authentication

- Functionality tested: Authentication protection of /api/me.
- Input: HTTP GET request without an authentication token.
- Testing procedure: Access the authenticated profile endpoint without providing credentials.
- Expected output: HTTP status 401.
- Validation: Verify that unauthenticated access is rejected.

Test Case F8 – Profile Endpoint Returns Authenticated Profile

- Functionality tested: Authenticated user profile retrieval.
- Input: HTTP GET request to /api/me with a valid authentication token.
- Testing procedure: Access the profile endpoint using the token generated during registration.
- Expected output: HTTP status 200 and the registered user's email address.
- Validation: Verify that the returned email matches functional@test.local.

Test Case F9 – Project Creation and Listing

- Functionality tested: Project creation and retrieval.
- Input: Project name Functional Project.
- Testing procedure: Create a project using the authenticated API and subsequently retrieve the project list.
- Expected output: Project creation returns HTTP 201, and the listing returns HTTP 200 with the newly created project.
- Validation: Verify that the created project's ID is present in the retrieved project list.

Test Case F10 – Project Creation Rejects Empty Name

- Functionality tested: Project name validation.
- Input: Project name containing only whitespace.
- Testing procedure: Submit a project creation request with an empty or whitespace-only name.
- Expected output: HTTP status 400.
- Validation: Verify that the invalid project creation request is rejected.

Test Case F11 – Dataset Upload and Content Retrieval

- Functionality tested: Dataset upload and retrieval.
- Input: A valid classification CSV dataset and an associated project ID.
- Testing procedure: Create a project, upload the CSV dataset using the filename ../../safe.csv, and retrieve the uploaded dataset content.
- Expected output: HTTP status 201 for upload, sanitized filename safe.csv, and HTTP status 200 when retrieving the dataset content.
- Validation: Verify the filename and confirm that the retrieved CSV contains the expected headers age,income,segment.

Test Case F12 – Dataset Upload Rejects Invalid CSV

- Functionality tested: CSV validation during dataset upload.
- Input: Invalid single-column CSV content onecolumn\nx\n.
- Testing procedure: Attempt to upload a CSV file that does not satisfy the application's CSV validation requirements.
- Expected output: HTTP status 400 with an error response.
- Validation: Verify the response status and presence of the error field.

Test Case F13 – Dataset Upload Rejects Foreign or Missing Project

- Functionality tested: Project ownership and dataset upload validation.
- Input: Dataset upload request containing project ID 9999, which is not an accessible project for the authenticated user.
- Testing procedure: Attempt to upload a dataset against the specified project ID.
- Expected output: HTTP status 404.
- Validation: Verify that the upload request is rejected.

Test Case F14 – Logout Invalidates Session

- Functionality tested: User logout and session invalidation.
- Input: Authenticated logout request using a valid session token.
- Testing procedure: Perform logout and subsequently attempt to access the authenticated profile endpoint using the same token.

- Expected output: Logout returns HTTP 200, and the subsequent profile request returns HTTP 401.
- Validation: Verify that the previously valid session token is no longer accepted.

Test Case F15 – Frontend Root Is Served

- Functionality tested: Frontend application delivery.
- Input: HTTP GET request to the root endpoint /.
- Testing procedure: Send a request to the application root and examine the response status, content type, and response body.
- Expected output: HTTP status 200, HTML content type, and a response body containing ModelLab.
- Validation: Verify the status code, HTML content type, and presence of the application name in the response.

2.3 Functional Test Case Implementation

The functional testing implementation was developed using Python's unittest framework and organized within the test_functional.py file under the project's tests directory. The implementation uses the TestFunctionalApi test class to organize and execute individual API-level functional test cases.

The test implementation uses the ApiServer helper to send HTTP requests to different application endpoints and examine the corresponding responses. Each test case validates specific application functionality by checking HTTP status codes, response data, and relevant response headers.

A common setup procedure is implemented using the setUp() method, which registers a test user and obtains an authentication token before each test case. The tearDown() method closes the API server after each test execution to maintain an organized testing environment.

A total of 15 functional test cases have been implemented to validate the application's health endpoint, user registration, login, authentication, project management, dataset upload and retrieval, session invalidation, and frontend delivery.

The source-code implementation of these test cases is presented in the following figures.

```
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> $content = Get-Content ".\tests\test_functional.py"
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> $start = ($content | Select-String '^    def test_health_endpoin
t').LineNumber
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> $end = ($content | Select-String '^    def test_login_rejects_wr
ong_password').LineNumber
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> $content[($start - 1)..($end - 2)]
    def test_health_endpoint_reports_service_status(self):
        status, body, headers = self.api.request("GET", "/api/health")
        self.assertEqual(status, 200)
        self.assertEqual(body, {"status": "ok", "service": "ModelLab"})
        self.assertEqual(headers.get("Content-Type"), "application/json; charset=utf-8")

    def test_register_rejects_missing_name(self):
        status, body, _ = self.api.request("POST", "/api/register", {
            "name": "", "email": "x@test.local", "password": "Password123"
        })
        self.assertEqual(status, 400)
        self.assertIn("error", body)

    def test_register_rejects_invalid_email(self):
        status, _, _ = self.api.request("POST", "/api/register", {
            "name": "X", "email": "invalid", "password": "Password123"
        })
        self.assertEqual(status, 400)

    def test_register_rejects_short_password(self):
        status, _, _ = self.api.request("POST", "/api/register", {
            "name": "X", "email": "x2@test.local", "password": "short"
        })
        self.assertEqual(status, 400)

    def test_duplicate_registration_returns_conflict(self):
        status, _, _ = self.api.request("POST", "/api/register", {
            "name": "Duplicate", "email": "functional@test.local", "password": "Password123"
        })
        self.assertEqual(status, 409)
```

Figure 2.1: Python Implementation of Functional API Test Cases (F1–F5)

Explanation: The code verifies the application's health endpoint and user registration functionality. It checks service status, validates mandatory registration fields, rejects invalid email addresses and short passwords, and prevents duplicate user registration through appropriate HTTP response codes.

```
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> $content = Get-Content ".\tests\test_functional.py"
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> $start = ($content | Select-String '^    def test_login_rejects_
wrong_password').LineNumber
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> $end = ($content | Select-String '^    def test_dataset_upload_a
nd_content_retrieval').LineNumber
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> $content[($start - 1)..($end - 2)]
    def test_login_rejects_wrong_password(self):
        status, body, _ = self.api.request("POST", "/api/login", {
            "email": "functional@test.local", "password": "WrongPassword"
        })
        self.assertEqual(status, 401)
        self.assertIn("error", body)

    def test_me_requires_authentication(self):
        status, _, _ = self.api.request("GET", "/api/me")
        self.assertEqual(status, 401)

    def test_me_returns_authenticated_profile(self):
        status, body, _ = self.api.request("GET", "/api/me", token=self.token)
        self.assertEqual(status, 200)
        self.assertEqual(body["email"], "functional@test.local")

    def test_project_creation_and_listing(self):
        status, created, _ = self.api.request("POST", "/api/projects",
                                              {"name": "Functional Project"}, self.token)
        self.assertEqual(status, 201)
        status, projects, _ = self.api.request("GET", "/api/projects", token=self.token)
        self.assertEqual(status, 200)
        self.assertTrue(any(p["id"] == created["id"] for p in projects))

    def test_project_creation_rejects_empty_name(self):
        status, _, _ = self.api.request("POST", "/api/projects", {"name": " "}, self.token)
        self.assertEqual(status, 400)

PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> |
```

Figure 2.2: Python Implementation of Functional API Test Cases (F6–F10)
Explanation: The code validates user login and authentication functionality, including rejection of incorrect passwords and prevention of unauthorized profile access. It also verifies authenticated profile retrieval, successful project creation and listing, and rejection of project creation requests containing an empty name.

```
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> $content = Get-Content ".\tests\test_functional.py"
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> $start = ($content | Select-String '^    def test_dataset_upload
_and_content_retrieval').LineNumber
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> $end = ($content | Select-String '^if __name__').LineNumber
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> $content[($start - 1)..($end - 2)]
    def test_dataset_upload_and_content_retrieval(self):
        _, project, _ = self.api.request("POST", "/api/projects", {"name": "Dataset Project"}, self.token)
        status, dataset, _ = self.api.request("POST", "/api/datasets/upload", {
            "project_id": project["id"], "filename": "../../safe.csv", "csv": CLASSIFICATION_CSV
        }, self.token)
        self.assertEqual(status, 201)
        self.assertEqual(dataset["filename"], "safe.csv")
        status, content, _ = self.api.request("GET", f"/api/datasets/{dataset['id']}/content",
                                              token=self.token)
        self.assertEqual(status, 200)
        self.assertIn("age,income,segment", content["csv"])

    def test_dataset_upload_rejects_invalid_csv(self):
        _, project, _ = self.api.request("POST", "/api/projects", {"name": "Bad CSV Project"}, self.token)
        status, body, _ = self.api.request("POST", "/api/datasets/upload", {
            "project_id": project["id"], "filename": "bad.csv", "csv": "onecolumn\nx\n"
        }, self.token)
        self.assertEqual(status, 400)
        self.assertIn("error", body)

    def test_dataset_upload_rejects_foreign_or_missing_project(self):
        status, _, _ = self.api.request("POST", "/api/datasets/upload", {
            "project_id": 9999, "filename": "x.csv", "csv": CLASSIFICATION_CSV
        }, self.token)
        self.assertEqual(status, 404)

    def test_logout_invalidates_session(self):
        status, _, _ = self.api.request("POST", "/api/logout", {}, self.token)
        self.assertEqual(status, 200)
        status, _, _ = self.api.request("GET", "/api/me", token=self.token)
        self.assertEqual(status, 401)

    def test_frontend_root_is_served(self):
        status, body, headers = self.api.request("GET", "/")
        self.assertEqual(status, 200)
        self.assertIn("text/html", headers.get("Content-Type", ""))
        self.assertIn(b"ModelLab", body if isinstance(body, bytes) else str(body).encode())
```

Figure 2.3: Python Implementation of Functional API Test Cases (F11–F15)
Explanation: The code validates dataset uploading and content retrieval, rejection of invalid CSV files, project access validation during dataset uploads, session invalidation after logout, and successful delivery of the ModelLab frontend through the application root endpoint.

2.4 Functional Testing Execution Results

2.4.1 Functional Testing Execution

The functional testing module was executed independently using Python's unittest framework through the following command:
python -m unittest -v tests.test_functional

The execution covered all 15 functional test cases implemented in the test_functional.py module. These test cases validated the application's health endpoint, user registration, login, authentication, project management, dataset upload and retrieval, session invalidation, and frontend delivery.

```
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> python -m unittest -v tests.test_functional
test_dataset_upload_and_content_retrieval (tests.test_functional.TestFunctionalApi.test_dataset_upload_and_content_retrieval) ... [ModelLab] "POST /api/register HTTP/1.1" 201 -
[ModelLab] "POST /api/projects HTTP/1.1" 201 -
[ModelLab] "POST /api/datasets/upload HTTP/1.1" 201 -
[ModelLab] "GET /api/datasets/1/content HTTP/1.1" 200 -
ok
test_dataset_upload_rejects_foreign_or_missing_project (tests.test_functional.TestFunctionalApi.test_dataset_upload_rejects_foreign_or_missing_project) ... [ModelLab] "POST /api/register HTTP/1.1" 201 -
[ModelLab] "POST /api/datasets/upload HTTP/1.1" 404 -
C:\Python314\Lib\tempfile.py:484: ResourceWarning: Implicitly cleaning up <HTTPError 404: 'Not Found'>
  _warnings.warn(self.warn_message, ResourceWarning)
ok
test_dataset_upload_rejects_invalid_csv (tests.test_functional.TestFunctionalApi.test_dataset_upload_rejects_invalid_csv) ... [ModelLab] "POST /api/register HTTP/1.1" 201 -
[ModelLab] "POST /api/projects HTTP/1.1" 201 -
[ModelLab] "POST /api/datasets/upload HTTP/1.1" 400 -
C:\Python314\Lib\tempfile.py:484: ResourceWarning: Implicitly cleaning up <HTTPError 400: 'Bad Request'>
  _warnings.warn(self.warn_message, ResourceWarning)
ok
test_duplicate_registration_returns_conflict (tests.test_functional.TestFunctionalApi.test_duplicate_registration_returns_conflict) ... [ModelLab] "POST /api/register HTTP/1.1" 201 -
[ModelLab] "POST /api/register HTTP/1.1" 409 -
C:\Python314\Lib\tempfile.py:484: ResourceWarning: Implicitly cleaning up <HTTPError 409: 'Conflict'>
  _warnings.warn(self.warn_message, ResourceWarning)
ok
test_frontend_root_is_served (tests.test_functional.TestFunctionalApi.test_frontend_root_is_served) ... [ModelLab] "POST /api/register HTTP/1.1" 201 -
[ModelLab] "GET / HTTP/1.1" 200 -
ok
test_health_endpoint_reports_service_status (tests.test_functional.TestFunctionalApi.test_health_endpoint_reports_service_status) ... [ModelLab] "POST /api/register HTTP/1.1" 201 -
[ModelLab] "GET /api/health HTTP/1.1" 200 -
ok
test_login_rejects_wrong_password (tests.test_functional.TestFunctionalApi.test_login_rejects_wrong_password) ... [ModelLab] "POST /api/register HTTP/1.1" 201 -
[ModelLab] "POST /api/login HTTP/1.1" 401 -
C:\Python314\Lib\tempfile.py:484: ResourceWarning: Implicitly cleaning up <HTTPError 401: 'Unauthorized'>
  _warnings.warn(self.warn_message, ResourceWarning)
ok
test_logout_invalidates_session (tests.test_functional.TestFunctionalApi.test_logout_invalidates_session) ... [ModelLab] "POST /api/register HTTP/1.1" 201 -
[ModelLab] "POST /api/logout HTTP/1.1" 200 -
[ModelLab] "GET /api/me HTTP/1.1" 401 -
ok
test_me_requires_authentication (tests.test_functional.TestFunctionalApi.test_me_requires_authentication) ... [ModelLab] "POST /api/register HTTP/1.1" 201 -
[ModelLab] "GET /api/me HTTP/1.1" 401 -
ok
test_me_returns_authenticated_profile (tests.test_functional.TestFunctionalApi.test_me_returns_authenticated_profile) ... [ModelLab] "POST /api/register HTTP/1.1" 201 -
[ModelLab] "GET /api/me HTTP/1.1" 200 -
ok
test_project_creation_and_listing (tests.test_functional.TestFunctionalApi.test_project_creation_and_listing) ... [ModelLab] "POST /api/register HTTP/1.1" 201 -
[ModelLab] "POST /api/projects HTTP/1.1" 201 -
[ModelLab] "GET /api/projects HTTP/1.1" 200 -
ok
test_project_creation_rejects_empty_name (tests.test_functional.TestFunctionalApi.test_project_creation_rejects_empty_name) ... [ModelLab] "POST /api/register HTTP/1.1" 201 -
[ModelLab] "POST /api/projects HTTP/1.1" 400 -
ok
test_register_rejects_invalid_email (tests.test_functional.TestFunctionalApi.test_register_rejects_invalid_email) ... [ModelLab] "POST /api/register HTTP/1.1" 201 -
[ModelLab] "POST /api/register HTTP/1.1" 400 -
ok
test_register_rejects_missing_name (tests.test_functional.TestFunctionalApi.test_register_rejects_missing_name) ... [ModelLab] "POST /api/register HTTP/1.1" 201 -
[ModelLab] "POST /api/register HTTP/1.1" 400 -
ok
test_register_rejects_short_password (tests.test_functional.TestFunctionalApi.test_register_rejects_short_password) ... [ModelLab] "POST /api/register HTTP/1.1" 201 -
[ModelLab] "POST /api/register HTTP/1.1" 400 -
ok

----------------------------------------------------------------------
Ran 15 tests in 9.267s

OK
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final>
```

Figure 2.4: Functional Testing Execution Results Showing 15 Successfully Passed Test Cases

Explanation: The screenshot displays the execution of all 15 functional API test cases. Each test case is marked as ok, indicating that the actual application response matched the expected result. The output also displays the

HTTP requests, response status codes, and final execution summary. All 15 tests completed successfully in approximately 9.267 seconds, with zero failures and zero errors.

2.4.2 Functional Testing Results Summary

The results obtained from the functional testing execution are summarized below.

Evaluation Parameter	Result
Total test cases executed	15
Successfully passed	15
Failed test cases	0
Errors encountered	0
Execution time	9.267 seconds
Overall execution status	PASSED

2.4.3 Result Analysis

The functional testing execution achieved a 100% test-case pass rate.

The results demonstrate that the tested application features behaved according to their expected functionality.

- The health endpoint successfully reported the application service status.
- Registration and login validation correctly handled valid and invalid credentials.
- Authentication mechanisms correctly restricted unauthorized access.
- Project creation and listing operations returned the expected results.
- Dataset upload and retrieval operations successfully handled the tested CSV data.
- Invalid dataset uploads and inaccessible project references were rejected.
- Logout successfully invalidated the existing session.
- The frontend root endpoint successfully served the ModelLab application.

$$\text{Pass Rate} = \frac{15}{15} \times 100 = 100\%$$

2.4.4 Conclusion

The functional testing phase was completed successfully, with all 15 implemented test cases passing without failures or errors. The results demonstrate that the tested user-facing API features and application components performed according to their expected behaviour under the evaluated conditions.

This completes the functional testing phase of the ModelLab application.

3. INTEGRATION TESTING

3.1 Overview of Integration Test Cases

Integration Testing is performed to verify whether different modules and components of an application interact and work together correctly. It focuses on validating the communication, data flow, and relationships between interconnected components to identify errors that may occur during their interaction.

In ModelLab, integration testing focuses on validating the interaction between CSV parsing, data preprocessing, machine learning model execution, evaluation metrics generation, and database operations. It ensures that data is correctly transferred and processed across the different components of the application.

The test cases are organized in a dedicated Python test module named test_integration.py within the tests directory. This separation helps maintain a structured testing implementation and allows integration test cases to be executed independently.

A total of six integration test cases are implemented to examine the interaction between data processing and machine learning components, validate classification and regression workflows, verify input validation, and examine database consistency.

3.2 Detailed Integration Test Cases

The following six integration test cases are implemented to verify the interactions between the major components of ModelLab.

Test Case I1 – Classification Pipeline Generates Expected Metrics

- **Objective:** Verify the integration of CSV parsing, preprocessing, classification model execution, and evaluation metrics generation.
- **Input:** A valid classification CSV dataset, target column, and classification model configuration.
- **Testing procedure:** Process the classification dataset through the machine learning pipeline, perform preprocessing, train the selected classification model, and generate evaluation metrics.
- **Expected output:** Successful classification model execution and generation of classification evaluation metrics.
- **Validation:** Verify that the processing pipeline returns the expected classification results and evaluation metrics.

Test Case I2 – Regression Pipeline Generates Expected Metrics

- **Objective:** Verify the integration of CSV parsing, preprocessing, regression model execution, and evaluation metrics generation.
- **Input:** A valid regression CSV dataset, target column, and regression model configuration.
- **Testing procedure:** Process the regression dataset through the machine learning pipeline, perform preprocessing, train the selected regression model, and generate evaluation metrics.

- **Expected output:** Successful regression model execution and generation of MAE, RMSE, and R^2 metrics.
- **Validation:** Verify that the regression pipeline returns the expected evaluation metrics.

Test Case I3 – Invalid Target Column Is Rejected

- **Objective:** Verify that the machine learning processing pipeline validates the specified target column.
- **Input:** A dataset containing columns that do not include the specified target column.
- **Testing procedure:** Submit the dataset for model processing using an invalid or non-existent target column.
- **Expected output:** The processing operation is rejected with an appropriate validation error.
- **Validation:** Verify that the pipeline does not proceed with model training when the target column is unavailable.

Test Case I4 – Unsupported Task Type Is Rejected

- **Objective:** Verify that the processing pipeline accepts only supported machine learning task types.
- **Input:** A dataset and an unsupported machine learning task type.
- **Testing procedure:** Submit the dataset for processing using an unsupported task configuration.
- **Expected output:** The processing operation is rejected with an appropriate validation error.
- **Validation:** Verify that unsupported task types are not processed by the machine learning pipeline.

Test Case I5 – Preprocessing Configuration Is Integrated with Model Execution

- **Objective:** Verify the interaction between preprocessing configurations and machine learning model execution.
- **Input:** A valid dataset and preprocessing configuration containing the required data transformation settings.
- **Testing procedure:** Apply the configured preprocessing operations and execute the machine learning processing pipeline.
- **Expected output:** Successful processing of the dataset and execution of the selected machine learning model.
- **Validation:** Verify that the preprocessing configuration is incorporated into the processing workflow and that the pipeline produces the expected output.

Test Case I6 – Database Records Maintain Consistent Relationships

- **Objective:** Verify the integration and relationships between users, sessions, projects, datasets, and machine learning experiments in the database.
- **Input:** User, session, project, dataset, and experiment records created within an isolated database environment.

- **Testing procedure:** Create the required database records, execute a classification model, store the experiment details, and retrieve the associated information using SQL JOIN operations.
- **Expected output:** Successful database operations with correctly associated records across the related tables.
- **Validation:** Verify that the retrieved records maintain the expected relationships between users, sessions, projects, datasets, and experiments.

3.3 Integration Test Case Implementation

The integration testing implementation was developed using Python's unittest framework and organized within the test_integration.py file under the project's tests directory.
The implementation consists of two test classes. The TestDataToModelIntegration class contains five test cases that validate the interaction between CSV parsing, preprocessing configurations, machine learning model execution, and evaluation metrics generation.

The TestDatabaseIntegration class contains one test case that validates the relationships between users, sessions, projects, datasets, and experiments within the database.
The test implementation uses the isolated_database helper to provide an isolated database environment for database integration testing. It also uses predefined classification and regression CSV datasets to validate the machine learning processing pipeline.
A total of six integration test cases are implemented to verify data flow, component interaction, pipeline validation, and database consistency.
The source-code implementation of these test cases is presented in the following figures.

```
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> $content = Get-Content ".\tests\test_integration.py"
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> $start = ($content | Select-String '^class TestDataToModelIntegration').LineNumber
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> $end = ($content | Select-String '^class TestDatabaseIntegration').LineNumber
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> $content[($start - 1)..($end - 2)]
class TestDataToModelIntegration(unittest.TestCase):
    def test_csv_parser_to_classification_training_and_metrics(self):
        headers, rows = server.csv_data(CLASSIFICATION_CSV)
        self.assertEqual(headers, ["age", "income", "segment"])
        self.assertEqual(len(rows), 12)
        result = server.run_model(CLASSIFICATION_CSV, "segment", "classification",
                                  "Decision Tree", 0.25, "mean", "onehot", "standard", 7)
        self.assertEqual(result["task"], "classification")
        self.assertIn("accuracy", result["metrics"])
        self.assertIn("confusion_matrix", result["metrics"])
        self.assertEqual(result["train_rows"] + result["test_rows"], 12)
        self.assertEqual(result["feature_count"], 2)
        self.assertTrue(result["predictions"])

    def test_csv_parser_to_regression_training_and_metrics(self):
        headers, rows = server.csv_data(REGRESSION_CSV)
        self.assertEqual(len(headers), 3)
        self.assertEqual(len(rows), 12)
        result = server.run_model(REGRESSION_CSV, "price", "regression",
                                  "Linear Regression", 0.25, "mean", "onehot", "standard", 7)
        self.assertEqual(result["task"], "regression")
        self.assertTrue({"mae", "rmse", "r2"} <= result["metrics"].keys())
        self.assertEqual(result["train_rows"] + result["test_rows"], 12)

    def test_preprocessing_options_work_in_pipeline(self):
        result = server.run_model(CLASSIFICATION_CSV, "segment", "classification",
                                  "Random Forest", 0.25, "mean", "label", "minmax", 11)
        self.assertEqual(result["config"]["encoding"], "label")
        self.assertEqual(result["config"]["scaling"], "minmax")

    def test_invalid_target_stops_pipeline_before_training(self):
        with self.assertRaisesRegex(ValueError, "Target column"):
            server.run_model(CLASSIFICATION_CSV, "missing", "classification",
                             "Decision Tree", 0.25, "mean", "onehot", "standard")

    def test_invalid_task_stops_pipeline(self):
        with self.assertRaisesRegex(ValueError, "Task must be"):
            server.run_model(CLASSIFICATION_CSV, "segment", "clustering",
                             "Decision Tree", 0.25, "mean", "onehot", "standard")

PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final>
```

Figure 3.1: Python Implementation of Data-to-Model Integration Test Cases (I1–I5)
Explanation: The code implements five integration test cases to validate the interaction between CSV parsing, classification and regression model training, preprocessing configurations, and evaluation metrics. It also verifies that invalid target columns and unsupported machine learning task types are correctly rejected by the processing pipeline.

```
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> $content = Get-Content ".\tests\test_integration.py"
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> $start = ($content | Select-String '^class TestDatabaseIntegrati
on').LineNumber
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> $content[($start - 1)..($content.Length - 1)]
class TestDatabaseIntegration(unittest.TestCase):
    def test_user_session_project_dataset_and_experiment_relations(self):
        with isolated_database() as (_, uploads):
            password = server.hash_pw("IntegrationPassword")
            with server.connect() as conn:
                uid = conn.execute(
                    "INSERT INTO users(name,email,pw_hash,created) VALUES(?,?,?,?)",
                    ("Integration", "integration@test.local", password, server.now())
                ).lastrowid
                conn.execute("INSERT INTO sessions(token,user_id,created) VALUES(?,?,?)",
                             ("token-1", uid, server.now()))
                pid = conn.execute(
                    "INSERT INTO projects(user_id,name,description,created) VALUES(?,?,?,?)",
                    (uid, "Integrated Project", "test", server.now())
                ).lastrowid
                csv_path = uploads / "dataset.csv"
                csv_path.write_text(CLASSIFICATION_CSV, encoding="utf-8")
                did = conn.execute(
                    """INSERT INTO datasets(user_id,project_id,filename,path,rows_count,columns_count,created)
                       VALUES(?,?,?,?,?,?,?)""",
                    (uid, pid, "dataset.csv", str(csv_path), 12, 3, server.now())
                ).lastrowid
                result = server.run_model(CLASSIFICATION_CSV, "segment", "classification",
                                          "Decision Tree", .25, "mean", "onehot", "standard")
                conn.execute(
                    """INSERT INTO experiments(user_id,project_id,dataset_id,model,task,target,metrics,config,created)
                       VALUES(?,?,?,?,?,?,?,?,?)""",
                    (uid, pid, did, result["model"], result["task"], result["target"],
                     __import__("json").dumps(result["metrics"]),
                     __import__("json").dumps(result["config"]), server.now())
                )
            with server.connect() as conn:
                row = conn.execute(
                    """SELECT u.email,p.name,d.filename,e.model,e.task
                       FROM users u JOIN projects p ON p.user_id=u.id
                       JOIN datasets d ON d.project_id=p.id
                       JOIN experiments e ON e.dataset_id=d.id WHERE u.id=?""", (uid,)
                ).fetchone()
            self.assertEqual(tuple(row), ("integration@test.local", "Integrated Project",
                                          "dataset.csv", "Decision Tree", "classification"))


if __name__ == "__main__":
    unittest.main(verbosity=2)
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final>
```

Figure 3.2: Python Implementation of Database Integration Test Case (I6)
Explanation: The code verifies the integration and relationships between users, sessions, projects, datasets, and machine learning experiments. It creates the required records in an isolated database, executes a classification model, stores the experiment details, and retrieves the associated information through SQL JOIN operations to validate database consistency.

3.4 Integration Testing Execution Results

3.4.1 Integration Testing Execution

The integration testing module was executed independently using Python's unittest framework through the following command: python -m unittest -v tests.test_integration

The execution covered all six integration test cases implemented in the test_integration.py module. These test cases validated the interaction between CSV parsing, data preprocessing, classification and regression model training, evaluation metrics generation, and database operations.

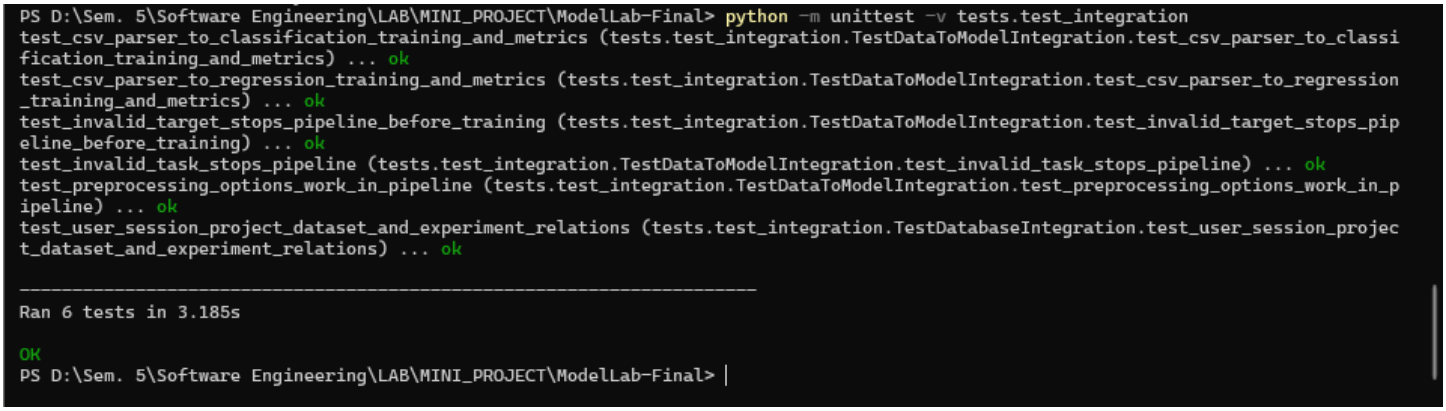


Figure 3.3: Integration Testing Execution Results Showing 6 Successfully Passed Test Cases

Explanation: The screenshot displays the execution of all six integration test cases. Each test case is marked as ok, indicating that the integrated components produced the expected results. The execution completed in approximately 3.185 seconds, with six tests passed, zero failures, and zero errors.

3.4.2 Integration Testing Results Summary

The results obtained from the integration testing execution are summarized below.

Evaluation Parameter	Result
Total test cases executed	6
Successfully passed	6
Failed test cases	0
Errors encountered	0
Execution time	3.185 seconds
Overall execution status	PASSED

3.4.3 Result Analysis

The integration testing execution achieved a 100% test-case pass rate.

The results demonstrate that the tested application components interacted and exchanged data successfully.

- CSV parsing successfully supplied structured data to the classification and regression pipelines.
- Classification and regression models executed successfully and generated their respective evaluation metrics.
- Preprocessing configurations were correctly incorporated into the machine learning pipeline.
- Invalid target columns and unsupported task types were rejected as expected.
- Database operations successfully maintained relationships between users, sessions, projects, datasets, and experiments.
- SQL JOIN operations successfully retrieved the associated records from the integrated database tables.

$$\text{Pass Rate} = \frac{6}{6} \times 100 = 100\%$$

3.4.4 Conclusion

The integration testing phase was completed successfully, with all six implemented test cases passing without failures or errors. The results demonstrate that the tested ModelLab components interacted correctly, supporting the proper flow of data through the machine learning processing pipeline and database operations.
This completes the integration testing phase of the ModelLab application.

Integration Testing – Completed

We have now completed the first two testing categories of the System Testing Report.

Testing category	Test cases	Passed	Failed
Functional / Component	15	15	0
Integration	6	6	0
Total	21	21	0

4. SYSTEM TESTING

4.1 Overview of System Test Cases

System Testing is performed to verify the behaviour of the complete integrated application as a single system. It evaluates whether the application satisfies its overall functional requirements and whether its different components work together correctly under realistic usage scenarios.

In ModelLab, system testing focuses on validating complete application workflows involving user authentication, project management, dataset uploading, machine learning experiment execution, result retrieval, report generation, and access control.

The system testing implementation is organized in the test_system.py file within the project's tests directory. It contains the TestCompleteSystemWorkflows test class, which uses the ApiServer helper to interact with the application through its API endpoints.

A total of four system test cases are implemented to validate complete classification and regression workflows, user-level project access restrictions, and dataset ownership requirements during experiment execution.

4.2 Detailed System Test Cases

Test Case S1 – Complete Classification Workflow and Report

- Objective: Verify the complete classification workflow from user authentication to report generation.
- Input: Authenticated user, project named Churn Analysis, classification CSV dataset, target column segment, and Decision Tree classification model.
- Testing procedure:
    1. Authenticate the registered user.
    2. Create a project.
    3. Upload the classification dataset.
    4. Execute a classification experiment.
    5. Retrieve the experiment details.
    6. Generate a report using the experiment ID.
- Expected output: Successful execution of all workflow operations, classification accuracy metrics, successful experiment retrieval, and a generated report containing the classification experiment.
- Validation: Verify HTTP response codes, the presence of accuracy metrics, experiment availability, and report contents.

Test Case S2 – Complete Regression Workflow

- Objective: Verify the complete regression workflow through the application.
- Input: Regression CSV dataset, project named Price Prediction, target column price, and Linear Regression model.
- Testing procedure: Create a project, upload the regression dataset, and execute a regression experiment.
- Expected output: HTTP status 201 and the regression evaluation metrics MAE, RMSE, and R^2.
- Validation: Verify successful experiment execution and the presence of all three regression metrics.

Test Case S3 – User Cannot Access Another User's Project

- Objective: Verify user-level project access restrictions.
- Input: Two independently registered users, a project owned by the second user, and the authentication token of the first user.

- Testing procedure: Register another user, create a private project using that user's token, and attempt to upload a dataset to that project using the original user's token.
- Expected output: HTTP status 404.
- Validation: Verify that a user cannot access another user's project through the dataset upload operation.

Test Case S4 – Experiment Requires Owned Dataset

- Objective: Verify that experiment execution requires an accessible and owned dataset.
- Input: Experiment request containing a non-existent dataset ID (9999) and project ID (0).
- Testing procedure: Submit an experiment execution request using the invalid dataset and project references.
- Expected output: HTTP status 404 with an error response.
- Validation: Verify that the application rejects the request instead of executing the experiment.

4.3 System Test Case Implementation

The system testing implementation was developed using Python's unittest framework and organized within the test_system.py file under the project's tests directory.

The implementation uses the TestCompleteSystemWorkflows class to organize four system-level test cases. The setUp() method registers a test user and obtains an authentication token before each test execution. The tearDown() method closes the API server after the test is completed.

The test cases interact with multiple application endpoints to validate complete workflows involving user authentication, project creation, dataset uploading, experiment execution, experiment retrieval, and report generation.

The implementation also verifies application-level access restrictions by checking whether users can access resources belonging to other users and whether experiment execution is restricted to valid, accessible datasets. A total of four system test cases are implemented to evaluate complete classification and regression workflows, resource ownership restrictions, and application-level validation.

The source-code implementation of these test cases is presented in the following figures.

```
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> $content = Get-Content ".\tests\test_system.py"
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> $start = ($content | Select-String '^    def test_complete_class
ification_workflow').LineNumber
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> $end = ($content | Select-String '^    def test_user_cannot_acce
ss_another_users_project').LineNumber
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> $content[($start - 1)..($end - 2)]
    def test_complete_classification_workflow_and_report(self):
        # Register â†' authenticate â†' create project â†' upload CSV â†' train â†' retrieve â†' report.
        status, me, _ = self.api.request("GET", "/api/me", token=self.token)
        self.assertEqual(status, 200)
        status, project, _ = self.api.request("POST", "/api/projects",
                                              {"name": "Churn Analysis", "description": "System test"},
                                              self.token)
        self.assertEqual(status, 201)
        status, dataset, _ = self.api.request("POST", "/api/datasets/upload", {
            "project_id": project["id"], "filename": "churn.csv", "csv": CLASSIFICATION_CSV
        }, self.token)
        self.assertEqual(status, 201)
        status, experiment, _ = self.api.request("POST", "/api/experiments/run", {
            "project_id": project["id"], "dataset_id": dataset["id"], "target": "segment",
            "task": "classification", "model": "Decision Tree", "test_size": 0.25,
            "missing": "mean", "encoding": "onehot", "scaling": "standard"
        }, self.token)
        self.assertEqual(status, 201)
        self.assertIn("accuracy", experiment["metrics"])
        status, experiments, _ = self.api.request("GET", "/api/experiments", token=self.token)
        self.assertEqual(status, 200)
        self.assertTrue(any(x["id"] == experiment["id"] for x in experiments))
        status, report, _ = self.api.request("POST", "/api/reports",
                                             {"experiment_ids": [experiment["id"]]}, self.token)
        self.assertEqual(status, 200)
        self.assertEqual(len(report["experiments"]), 1)
        self.assertEqual(report["experiments"][0]["task"], "classification")

    def test_complete_regression_workflow(self):
        _, project, _ = self.api.request("POST", "/api/projects",
                                         {"name": "Price Prediction"}, self.token)
        _, dataset, _ = self.api.request("POST", "/api/datasets/upload", {
            "project_id": project["id"], "filename": "prices.csv", "csv": REGRESSION_CSV
        }, self.token)
        status, result, _ = self.api.request("POST", "/api/experiments/run", {
            "project_id": project["id"], "dataset_id": dataset["id"], "target": "price",
            "task": "regression", "model": "Linear Regression"
        }, self.token)
        self.assertEqual(status, 201)
        self.assertTrue({"mae", "rmse", "r2"} <= result["metrics"].keys())

PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final>
```

Figure 4.1: Python Implementation of Complete Classification and Regression System Test Cases (S1–S2)

Explanation: The code implements two system-level test cases to validate complete machine learning workflows within ModelLab. The classification test verifies user authentication, project creation, dataset uploading, model execution, experiment retrieval, and report generation. The regression test verifies project creation, dataset uploading, regression model execution, and generation of MAE, RMSE, and R^2 evaluation metrics.

```
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> $content = Get-Content ".\tests\test_system.py"
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> $start = ($content | Select-String '^    def test_user_cannot_ac
cess_another_users_project').LineNumber
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> $end = ($content | Select-String '^if __name__').LineNumber
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> $content[($start - 1)..($end - 2)]
    def test_user_cannot_access_another_users_project(self):
        _, other, _ = self.api.request("POST", "/api/register", {
            "name": "Other", "email": "other@test.local", "password": "OtherPass123"
        })
        other_token = other["token"]
        _, other_project, _ = self.api.request(
            "POST", "/api/projects", {"name": "Private Project"}, other_token
        )
        status, _, _ = self.api.request("POST", "/api/datasets/upload", {
            "project_id": other_project["id"], "filename": "x.csv", "csv": CLASSIFICATION_CSV
        }, self.token)
        self.assertEqual(status, 404)

    def test_experiment_requires_owned_dataset(self):
        status, body, _ = self.api.request("POST", "/api/experiments/run", {
            "dataset_id": 9999, "project_id": 0, "target": "segment",
            "task": "classification", "model": "Decision Tree"
        }, self.token)
        self.assertEqual(status, 404)
        self.assertIn("error", body)


PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final>
```

Figure 4.2: Python Implementation of Access Control and Dataset Ownership System Test Cases (S3–S4)
Explanation: The code verifies application-level resource access restrictions. It checks that one user cannot upload datasets to another user's private project and that experiment execution is rejected when the specified dataset does not exist or is not accessible to the authenticated user.

4.4 System Testing Execution Results

4.4.1 System Testing Execution

The system testing module was executed independently using Python's unittest framework through the following command: python -m unittest -v tests.test_system

The execution covered all four system test cases implemented in the test_system.py module. These test cases evaluated complete classification and regression workflows, including user authentication, project creation, dataset uploading, machine learning experiment execution, experiment retrieval, and report generation. They also verified application-level access restrictions and dataset ownership validation.

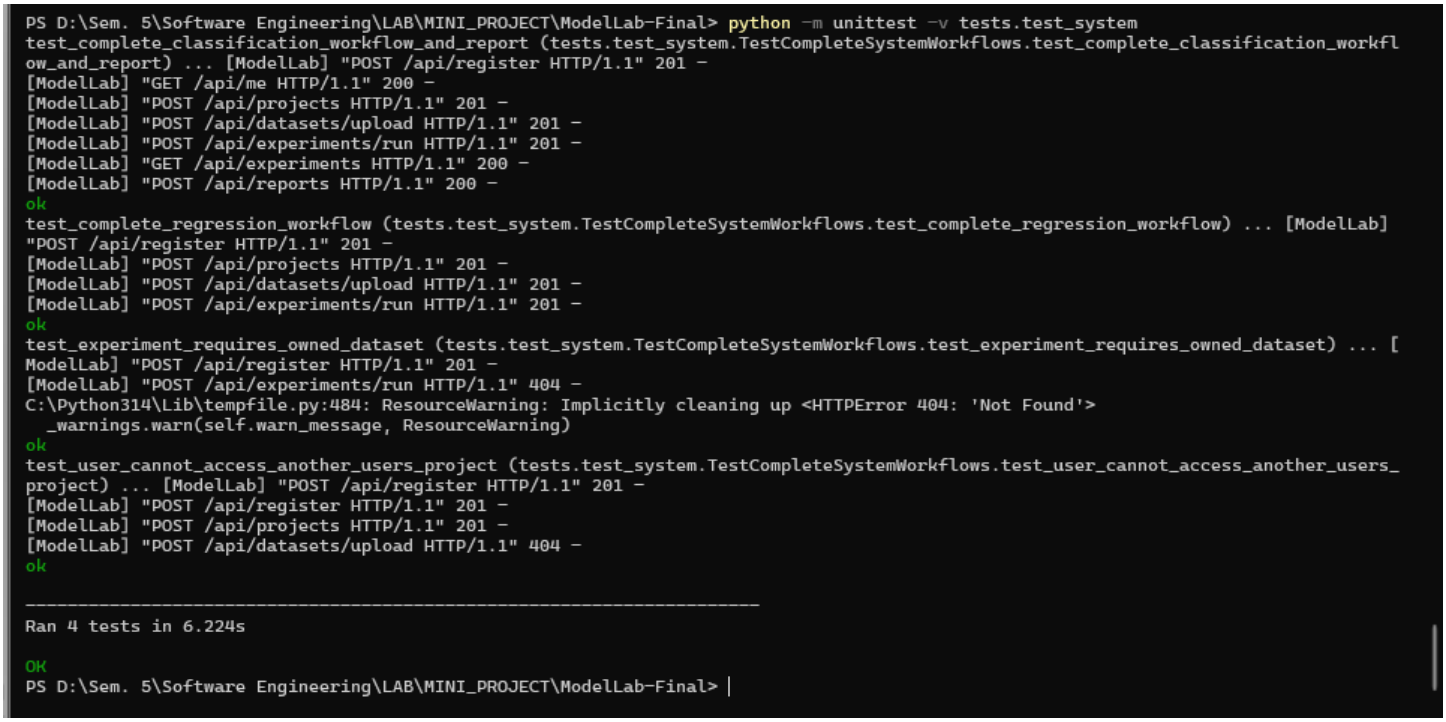


Figure 4.3: System Testing Execution Results Showing 4 Successfully Passed Test Cases

Explanation: The screenshot displays the execution of all four system test cases. Each test case is marked as ok, indicating that the complete application workflows and access-control validations produced the expected results. The execution completed in approximately 6.224 seconds, with four tests passed, zero failures, and zero errors.

4.4.2 System Testing Results Summary

The results obtained from the system testing execution are summarized below.

Evaluation Parameter	Result
Total test cases executed	4
Successfully passed	4
Failed test cases	0
Errors encountered	0
Execution time	6.224 seconds
Overall execution status	PASSED

4.4.3 Result Analysis

The system testing execution achieved a 100% test-case pass rate.
The results demonstrate that the complete ModelLab application performed according to the expected behaviour under the tested scenarios.

- The classification workflow successfully completed authentication, project creation, dataset uploading, model execution, experiment retrieval, and report generation.
- The regression workflow successfully executed the Linear Regression experiment and generated the required evaluation metrics.
- User-level access restrictions correctly prevented unauthorized access to another user's project.
- Experiment execution correctly rejected requests containing an invalid or inaccessible dataset reference.

$$\text{Pass Rate} = \frac{4}{4} \times 100 = 100\%$$

4.4.4 Conclusion

The system testing phase was completed successfully, with all four implemented test cases passing without failures or errors. The results demonstrate that the tested end-to-end application workflows and access-control mechanisms operated according to their expected behaviour.

This completes the system testing phase of the ModelLab application.

Testing category	Tests	Passed	Failed
Functional / Component	15	15	0
Integration	6	6	0
System	4	4	0
End-to-End	2	2	0
Overall	27	27	0

5. END-TO-END TESTING

5.1 Overview of End-to-End Test Cases

End-to-End Testing is performed to verify the complete workflow of an application from the user's initial interaction to the final expected outcome. It ensures that the different application components work together correctly throughout an entire user journey.

In ModelLab, end-to-end testing focuses on validating complete application workflows involving service availability, frontend resource delivery, user registration, authentication, project creation, dataset uploading, machine learning experiment execution, and experiment history retrieval.

The implementation is organized in the test_end_to_end.py file within the project's tests directory. It uses the TestEndToEnd class and the ApiServer helper to interact with the application through its HTTP endpoints. A total of two end-to-end test cases are implemented to verify frontend and backend availability and the complete user journey from registration to experiment history.

5.2 Detailed End-to-End Test Cases

Test Case E1 – Health and Frontend Assets Are Reachable

- Objective: Verify that the application service and its frontend resources are accessible.
- Input: HTTP GET requests to /api/health, /, /static/app.js, and /static/style.css.
- Testing procedure:
    1. Access the application health endpoint.
    2. Access the frontend root endpoint.
    3. Retrieve the JavaScript application file.
    4. Retrieve the CSS stylesheet.
- Expected output: All endpoints return HTTP status 200. The health endpoint reports ok, the frontend contains ModelLab, and the JavaScript and CSS resources contain content.
- Validation: Verify the HTTP status codes, health response, frontend content, HTML content type, and non-empty JavaScript and CSS responses.

Test Case E2 – User Journey from Registration to Experiment History

- Objective: Verify the complete user journey from account registration to retrieving machine learning experiment history.
- Input: User registration details, project name Journey, classification CSV dataset, target column segment, and Random Forest classification model.
- Testing procedure:
    1. Register a new user.
    2. Log in using the registered credentials.
    3. Create a project.
    4. Upload the classification dataset.
    5. Execute a classification experiment using the Random Forest model.
    6. Retrieve the experiment history.
- Expected output: Successful registration, login, project creation, dataset upload, experiment execution, and experiment history retrieval.
- Validation: Verify the corresponding HTTP response codes and confirm that the retrieved experiment history contains the ID of the newly executed experiment.

5.3 End-to-End Test Case Implementation

The end-to-end testing implementation was developed using Python's unittest framework and organized within the test_end_to_end.py file under the project's tests directory.

The implementation uses the TestEndToEnd class to organize two test cases. The setUp() method initializes the API server before each test execution, while the tearDown() method closes the API server after the test is completed.

The first test case validates the availability of the application health endpoint and frontend resources, including the HTML page, JavaScript file, and CSS stylesheet.

The second test case validates a complete user journey involving registration, login, project creation, dataset uploading, machine learning experiment execution, and experiment history retrieval.

A total of two end-to-end test cases are implemented to verify application accessibility and complete user workflows.

The source-code implementation of these test cases is presented in the following figure.

```
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> Get-Content ".\tests\test_end_to_end.py"
"""End-to-end smoke tests across HTTP server, frontend delivery and API workflows."""
import unittest
from tests.helpers import ApiServer, CLASSIFICATION_CSV


class TestEndToEnd(unittest.TestCase):
    def setUp(self):
        self.api = ApiServer()

    def tearDown(self):
        self.api.close()

    def test_health_and_frontend_assets_are_reachable(self):
        status, health, _ = self.api.request("GET", "/api/health")
        self.assertEqual(status, 200)
        self.assertEqual(health["status"], "ok")
        status, html, headers = self.api.request("GET", "/")
        self.assertEqual(status, 200)
        self.assertIn(b"ModelLab", html)
        self.assertIn("text/html", headers.get("Content-Type", ""))
        # Frontend JS and CSS are delivered by the server's static route.
        status, js, _ = self.api.request("GET", "/static/app.js")
        self.assertEqual(status, 200)
        self.assertGreater(len(js), 0)
        status, css, _ = self.api.request("GET", "/static/style.css")
        self.assertEqual(status, 200)
        self.assertGreater(len(css), 0)

    def test_user_journey_from_registration_to_experiment_history(self):
        status, registered, _ = self.api.request("POST", "/api/register", {
            "name": "Journey User", "email": "journey@test.local", "password": "JourneyPass123"
        })
        self.assertEqual(status, 201)
        token = registered["token"]
        status, login, _ = self.api.request("POST", "/api/login", {
            "email": "journey@test.local", "password": "JourneyPass123"
        })
        self.assertEqual(status, 200)
        token = login["token"]
        _, project, _ = self.api.request("POST", "/api/projects", {"name": "Journey"}, token)
        _, dataset, _ = self.api.request("POST", "/api/datasets/upload", {
            "project_id": project["id"], "filename": "journey.csv", "csv": CLASSIFICATION_CSV
        }, token)
        status, experiment, _ = self.api.request("POST", "/api/experiments/run", {
            "project_id": project["id"], "dataset_id": dataset["id"], "target": "segment",
            "task": "classification", "model": "Random Forest"
        }, token)
        self.assertEqual(status, 201)
        status, history, _ = self.api.request("GET", "/api/experiments", token=token)
        self.assertEqual(status, 200)
        self.assertEqual(history[0]["id"], experiment["id"])


if __name__ == "__main__":
    unittest.main(verbosity=2)
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> |
```

Figure 5.1: Python Implementation of End-to-End Test Cases (E1–E2)
Explanation: The code implements two end-to-end test cases. The first verifies the accessibility of the application health endpoint and frontend resources, including HTML, JavaScript, and CSS files. The second validates the complete user journey from registration and login through project creation, dataset uploading, Random Forest classification experiment execution, and experiment history retrieval.

5.4 End-to-End Testing Execution Results

5.4.1 End-to-End Testing Execution

The end-to-end testing module was executed independently using Python's unittest framework through the following command: python -m unittest -v tests.test_end_to_end

The execution covered both end-to-end test cases implemented in the test_end_to_end.py module. These test cases validated application service availability, frontend resource accessibility, and the complete user journey from registration and authentication to project creation, dataset uploading, machine learning experiment execution, and experiment history retrieval.

```
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final> python -m unittest -v tests.test_end_to_end
test_health_and_frontend_assets_are_reachable (tests.test_end_to_end.TestEndToEnd.test_health_and_frontend_assets_are_reachable) ..
. [ModelLab] "GET /api/health HTTP/1.1" 200 -
[ModelLab] "GET / HTTP/1.1" 200 -
[ModelLab] "GET /static/app.js HTTP/1.1" 200 -
[ModelLab] "GET /static/style.css HTTP/1.1" 200 -
ok
test_user_journey_from_registration_to_experiment_history (tests.test_end_to_end.TestEndToEnd.test_user_journey_from_registration_t
o_experiment_history) ... [ModelLab] "POST /api/register HTTP/1.1" 201 -
[ModelLab] "POST /api/login HTTP/1.1" 200 -
[ModelLab] "POST /api/projects HTTP/1.1" 201 -
[ModelLab] "POST /api/datasets/upload HTTP/1.1" 201 -
[ModelLab] "POST /api/experiments/run HTTP/1.1" 201 -
[ModelLab] "GET /api/experiments HTTP/1.1" 200 -
ok

----------------------------------------------------------------------
Ran 2 tests in 3.587s

OK
PS D:\Sem. 5\Software Engineering\LAB\MINI_PROJECT\ModelLab-Final>
```

Figure 5.2: End-to-End Testing Execution Results Showing 2 Successfully Passed Test Cases
Explanation: The screenshot displays the execution of both end-to-end test cases. Each test case is marked as ok, indicating that the tested application workflows completed successfully. The execution completed in approximately 3.587 seconds, with two tests passed, zero failures, and zero errors. The HTTP request logs further demonstrate the successful interaction with the application endpoints and frontend resources.

5.4.2 End-to-End Testing Results Summary

Evaluation Parameter	Result
Total test cases executed	2
Successfully passed	2
Failed test cases	0
Errors encountered	0
Execution time	3.587 seconds
Overall execution status	PASSED

5.4.3 Result Analysis

The end-to-end testing execution achieved a 100% test-case pass rate.
The results demonstrate that the tested complete application workflows operated according to their expected behaviour.

- The application health endpoint successfully reported the service status.
- The frontend HTML page, JavaScript application, and CSS stylesheet were successfully accessible.
- User registration and login operations completed successfully.
- Project creation and dataset uploading were successfully performed.
- The Random Forest classification experiment executed successfully.
- The experiment history endpoint successfully retrieved the executed experiment.

$$\text{Pass Rate} = \frac{2}{2} \times 100 = 100\%$$

5.4.4 Conclusion

The end-to-end testing phase was completed successfully, with both implemented test cases passing without failures or errors. The results demonstrate that the tested user workflows, frontend resources, and backend API operations worked together successfully throughout the evaluated application journeys.
This completes the end-to-end testing phase of the ModelLab application.

6. Overall System Testing Conclusion

The system testing report evaluated the ModelLab application through four testing categories: Functional / Component Testing, Integration Testing, System Testing, and End-to-End Testing.

A total of 27 test cases were executed across these four categories, and all 27 test cases passed successfully.

Testing category	Total tests	Passed	Failed
Functional / Component	15	15	0
Integration	6	6	0
System	4	4	0
End-to-End	2	2	0
Overall	27	27	0

100%
Overall test-case pass rate
27 passed out of 27 executed test cases

The successful execution of these testing activities demonstrates that the tested ModelLab functionalities, component interactions, complete application workflows, and user journeys behaved according to their expected results.

The testing activities provide evidence of the application's functional correctness and the integration of its major components under the evaluated scenarios.

Approvals

Role	Name	Signature / Email	Date
Project Team Member	**Pratham Kumar** — PES2UG24CS368	prathamkumar041104@gmail.com	11-11-2026
Project Team Member	**Nirupama Jayaraman** — PES2UG24CS324	jnirupama06@gmail.com	11-11-2026
Project Team Member	**Shreyas Bellad** — PES2UG25CS824	shreyasbellad2006@gmail.com	11-11-2026
Project Team Member	**Parinitha Acharya** — PES2UG24CS339	parinitha.acharya2006@gmail.com	11-11-2026
Course Coordinator	**Sheela Devi M.**	sheelam@pes.edu	—

THANK_YOU